An Energy-Efficient Hardware Accelerator for Convolutional Neural Networks on Zynq SoC

1st Sayooj PV, Adith M, Bennison P Baby, Anandhakrishnan, Ms Deepa Prabhu, Ms Nayana PK
Department of Electronics and Communication Engineering
Muthoot Institute of Technology and Science
Kochi, India

***Abstract*—The deployment of Convolutional Neural Networks (CNNs) on edge devices is heavily constrained by power consumption, memory bandwidth, and computational latency. This paper presents a highly optimized, low-power CNN hardware accelerator deployed on the Xilinx Zynq-7000 SoC targeting the MNIST dataset. The proposed architecture minimizes external memory access through a custom Processing Element (PE) array and shift-buffer utilization. To overcome the dual-port BRAM bandwidth bottleneck, a novel sliding window buffer mechanism was implemented to enable continuous pixel streaming. Furthermore, we introduce a Sparsity-Aware Dynamic Clock Gating (SADCG) technique that leverages ReLU-induced sparsity to dynamically gate inactive Multiply-Accumulate (MAC) operations, significantly reducing dynamic power consumption. The design was synthesized in Vivado, co-verified using MATLAB, and achieved real-time classification with high resource efficiency and strict timing closure.**

***Index Terms*—CNN Accelerator, FPGA, Zynq-7000, Edge Computing, Dynamic Clock Gating, Hardware-Software Co-Design, Quantization, DSP48E1**

I. Introduction

Convolutional Neural Networks (CNNs) have achieved state-of-the-art results in various image classification, object detection, and natural language processing tasks. However, the remarkable accuracy of these deep learning models comes at a steep computational cost. Their high computational complexity and massive memory bandwidth requirements pose significant challenges for deployment on edge devices, which are often bound by strict power, area, and thermal constraints.

Historically, the industry has relied on cloud-based Artificial Intelligence (AI) diagnostics to bypass these edge constraints. While cloud computing offers virtually limitless floating-point performance, it suffers from critical latency bottlenecks, severe privacy risks, and a strict dependence on stable internet connectivity. These factors are entirely unacceptable for mission-critical applications, real-time autonomous systems, and remote biomedical monitoring.

To bridge this gap, the edge computing paradigm pushes inference directly to the local sensor node. Field-Programmable Gate Arrays (FPGAs) offer a promising solution for edge AI by providing flexible, highly parallel hardware fabrics that can be tailored to specific neural network architectures. Unlike general-purpose von Neumann processors that constantly fetch generic instructions from memory, FPGAs allow developers to build custom spatial datapaths that perfectly match the algorithmic dataflow of the CNN, thereby minimizing control overhead and maximizing throughput per watt.

This paper proposes a custom, energy-efficient hardware accelerator optimized for MNIST digit classification. We utilize the Xilinx Zynq Z7-20 SoC, effectively co-designing the hardware fabric (Programmable Logic, or PL) and software control (Processing System, or PS).

Our specific contributions to the field of edge AI accelerators include:

- **Memory-Optimized Datapath:** A customized sliding window buffer architecture that circumvents standard Block RAM (BRAM) port limitations, ensuring 100% utilization of the computing array without data starvation.
- **Circuit-Level Energy Optimization:** A Sparsity-Aware Dynamic Clock Gating (SADCG) logic module that detects zero-valued activations in real-time and suppresses dynamic switching power during redundant ReLU-induced zero-computations.
- **Hybrid Hardware-Software Co-Design:** A fully synthesizable Register-Transfer Level (RTL) implementation orchestrated by an ARM Cortex-A9 processor over an AXI interconnect, ensuring independent, real-time edge classification.

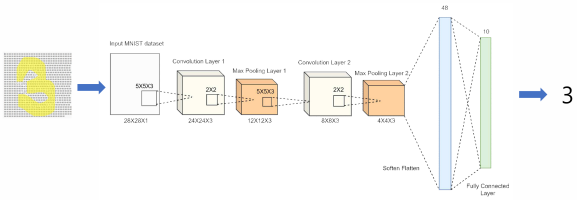


Fig. 1. Target CNN Model Structure optimized for MNIST classification, detailing the feature map dimensions at each layer.

II. Related Work

The rapid expansion of deep learning has catalyzed the development of various hardware acceleration paradigms. Understanding the tradeoffs between these platforms is critical for justifying an FPGA-based approach for edge AI.

A. GPU and ASIC Implementations

The training and deployment of Deep Neural Networks (DNNs) have traditionally relied on high-performance Graphics Processing Units (GPUs). While GPUs excel at massively

parallel floating-point operations due to their thousands of streaming multiprocessors, their high Thermal Design Power (TDP)—often exceeding 200W—makes them entirely unsuitable for battery-powered or heavily constrained edge devices.

Conversely, Application-Specific Integrated Circuits (ASICs) offer the highest energy efficiency and lowest computational latency by stripping away all unnecessary routing overhead. However, ASICs suffer from prolonged time-to-market, complete lack of post-silicon flexibility, and exorbitant non-recurring engineering (NRE) costs. If a CNN architecture is updated or a new activation function is adopted by the machine learning community, the rigid ASIC silicon immediately becomes obsolete.

B. FPGA-Based Accelerators

FPGAs serve as an optimal middle ground, offering hardware-level parallelism while maintaining post-deployment reconfigurability. Foundational works, such as the DianNao accelerator family [4] and various loop-unrolling strategies [2], have demonstrated the viability of spatial architectures on reconfigurable logic.

However, many of these classical designs still suffer from severe "memory walls" or BRAM bandwidth bottlenecks when streaming high-resolution feature maps. In standard FPGA architectures, Dual-Port BRAM can only service two memory read/write requests per clock cycle. When a convolution operation requires a 3×3 or 5×5 window of pixels simultaneously, the memory interface becomes the primary chokepoint, forcing the compute engines to stall [9].

Furthermore, while extensive research exists on reducing parameter precision—such as Binary Neural Networks (BNNs)—these aggressive quantization techniques often result in severe accuracy degradation [8]. Our work builds upon existing spatial architectures by directly addressing the memory bottleneck through a localized shift-register hierarchy and introducing dynamic power reduction via SADCG without altering the underlying precision or accuracy of the trained model.

III. Mathematical Framework and Data Quantization

To properly partition the hardware and software domains, it is necessary to mathematically define the operations governing the CNN pipeline and how they are translated into digital logic suitable for the FPGA fabric.

A. Feature Map Dimensionality

The accelerator is validated using the MNIST dataset, a standard benchmark comprising 60,000 training images and 10,000 testing images of handwritten digits (0-9). Each input image is a 28×28 grayscale matrix. The spatial dimensions of the output feature map O for any given convolutional layer can be predetermined using the following fundamental equation:

$$O = \frac{W - K + 2P}{S} + 1 \tag{1}$$

Where W is the input dimension, K is the kernel size, P is the zero-padding applied to the borders, and S is the stride of the convolution. By carefully selecting $K = 5$, $P = 0$, and $S = 1$ for our primary layers, we maintain deterministic routing within our hardware sliding windows.

B. Offline Training and Preprocessing

Prior to hardware streaming, the incoming images are subjected to Min-Max normalization. This scales the raw pixel intensities from an integer range of $[0, 255]$ to a normalized $[0, 1]$ dynamic range to improve the convergence rate during gradient descent. The network was trained offline utilizing the PyTorch framework, leveraging Cross-Entropy loss and the Adam optimizer over 20 epochs to ensure robust feature extraction.

C. Symmetric INT8 Post-Training Quantization (PTQ)

A fundamental challenge in FPGA edge deployment is that high-precision 32-bit floating-point (FP32) weights are highly incompatible with ultra-low-power embedded implementations. Standard IEEE-754 floating-point adders and multipliers consume excessive Look-Up Tables (LUTs) and DSP slices, severely limiting the number of parallel Processing Elements (PEs) that can fit on the silicon.

To bridge the gap between software simulation and hardware execution, we performed a symmetric INT8 Post-Training Quantization (PTQ) [3]. The floating-point synaptic weights (W_{FP}) and input activations are linearly scaled and saturated into 8-bit signed integers, yielding a strict dynamic range of $[-128, 127]$. The quantization mapping is mathematically defined as:

$$W_{INT8} = \text{clip}\left(\text{round}\left(\frac{W_{FP}}{S_W}\right), -128, 127\right) \tag{2}$$

Where S_W is the fractional scale factor derived from the maximum absolute value of the weight tensor. Bias values are quantized to 32-bit integers to prevent overflow during the deep accumulation stages. These quantized parameters are ultimately serialized into hexadecimal format ('.mem' files) to pre-initialize the Distributed BRAM of the Zynq SoC during the bitstream generation phase, entirely eliminating the need to load weights from an external SD card at runtime.

D. Convolutional DSP Mapping

The core computation of the accelerator is the 2D discrete convolution. For an input feature map X and a trained weight kernel W of size $K \times K$, the output feature map Y at spatial coordinates (m, n) is computed as:

$$Y[m,n] = \sum_{i=0}^{K-1}\sum_{j=0}^{K-1} X[m+i, n+j] \cdot W[i,j] + B \tag{3}$$

To maximize throughput, our PE array parallelizes the inner summation loops (i and j) by mapping them directly to the dedicated DSP48E1 slices available on the Zynq-7000 fabric [10]. Each DSP slice is specifically configured to utilize its internal 25-bit pre-adder and 25×18 two's-complement

multiplier to execute the MAC operations in a highly pipelined, single-cycle manner.

E. Non-Linearity and MaxPooling

The non-linear activation function chosen for this architecture is the Rectified Linear Unit (ReLU), mathematically defined as:

$$f(x) = \max(0, x) \tag{4}$$

From a hardware perspective, this is exceptionally efficient. Rather than executing a complex floating-point activation (like Sigmoid or Tanh, which require expensive Taylor series approximations or large Look-Up Tables), the RTL simply evaluates the sign bit (the Most Significant Bit, or MSB) of the two's-complement accumulated integer. If the sign bit is 1 (indicating a negative value), the output multiplexer rigidly drives the signal to zero. This natural, cost-free clipping induces the high sparsity that is subsequently leveraged by our SADCG architecture.

To achieve spatial invariance and aggressively downsample the feature maps, a 2×2 MaxPooling layer is applied with a stride of 2:

$$Y_{pool} = \max_{(i,j) \in P} (X[i, j]) \tag{5}$$

This operation employs a cascaded tree of digital comparators to reduce the dimensionality of the intermediate data by 75%, drastically reducing the requisite on-chip memory utilization for the subsequent fully connected layers.

IV. Proposed System Architecture

The core objective of the proposed architecture is to minimize memory access overhead, maximize computational throughput, and maintain strict deterministic timing while operating within a constrained thermal envelope.

A. Hardware-Software Co-Design via AXI Interconnect

The system architecture explicitly partitions the workload between the Zynq Processing System (PS) and the Programmable Logic (PL). The ARM Cortex-A9 processor (PS) is tasked with handling non-deterministic, high-level operations: managing the UART serial communication with the host PC, preprocessing the raw input data arrays, and dispatching control signals.

Data is passed to the PL via the Advanced eXtensible Interface (AXI). Unlike high-throughput, 4K video processing architectures that rely on complex Direct Memory Access (DMA) protocols, our system utilizes a Memory-Mapped AXI4-Lite interface for control registers and AXI GPIO for streamlined data transfer. Since the payload for a single 28×28 image inference is highly localized, configuring complex DMA scatter-gather lists introduces unnecessary software driver overhead and latency. A strict hardware handshake protocol utilizing `start`, `busy`, and `done` signals ensures cycle-accurate synchronization between the ARM software driver and the hardware pipeline without the need for an underlying Operating System.

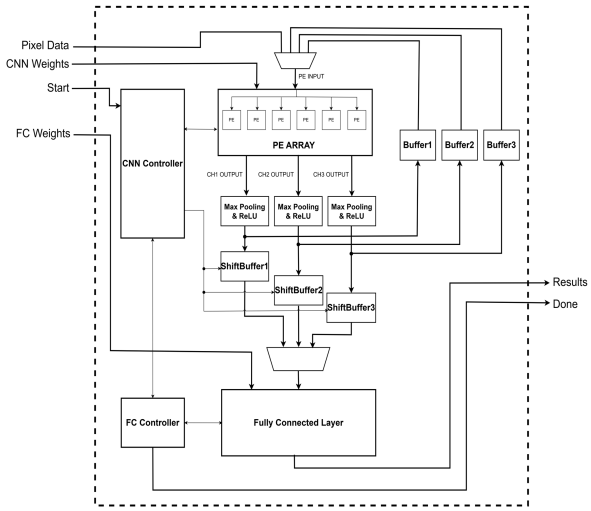


Fig. 2. Overall System Block Diagram of the Zynq-based Accelerator, showing the AXI boundaries between the PS and PL domains.

B. Finite State Machine (FSM) Controller

The overarching dataflow within the Programmable Logic is orchestrated by a master Finite State Machine (FSM). To guarantee deterministic execution, the FSM cycles through four primary states: `IDLE`, `LOAD_WEIGHTS`, `STREAM_PIXELS`, and `WRITEBACK`. The FSM specifically generates the addresses for the Block RAMs, manages the sliding window coordinate shifting, ensures the proper sequential fetching of the pre-trained weights, and controls the write-enable (WE) signals for the intermediate FIFO buffers bridging the PE arrays.

C. Processing Element (PE) Array and Memory Minimization

To drastically reduce dynamic power consumption, the design strictly limits external memory transactions. Fetching data from off-chip DDR3 memory requires orders of magnitude more energy than an on-chip SRAM read [8]. Therefore, all intermediate feature maps generated by the convolutional layers are kept entirely on-chip.

This is achieved by utilizing highly coupled FIFO buffers and localized Shift Buffers. The Shift Buffers act as customized L1 caches, managing the input data for the convolution operations. They drastically reduce redundant memory reads by maximizing the reuse of overlapping receptive fields as the convolution filter slides across the image. Once a pixel is fetched from BRAM, it cascades through the shift registers and is reused multiple times across different MAC operations before being discarded.

D. Sliding Window Buffer Optimization

As previously noted, a critical bottleneck in FPGA accelerators is the read-port limitation of standard Block RAMs. To overcome this starvation issue, a custom *Sliding Window* module was developed in Verilog.

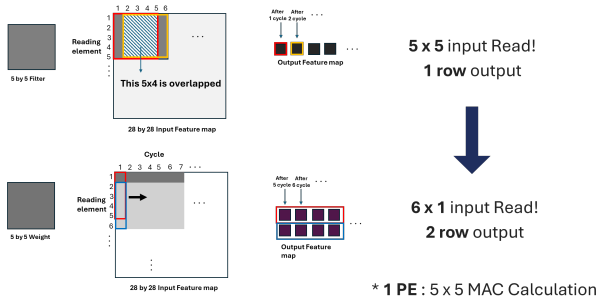


Fig. 3. Memory access minimization technique within the PE Array using localized shift buffers.

The incoming image data stream is pre-buffered into a 6×28 sliding window buffer (labeled `BUF_Slide`). Upon receiving a `slide_trigger` from the master FSM, the buffer shifts its entire contents upward by 2 rows. This hardware configuration effectively mimics a 2D sliding window effect, delivering an entire 6-row column of pixel data per clock cycle directly to the parallel convolution pipeline. This guarantees that the PE array is never starved for data, eliminating stall cycles and pushing the MAC utilization rate toward its theoretical maximum.

V. Innovative Energy Optimization: Sparsity-Aware Dynamic Clock Gating

While the shift buffers and localized FIFOs successfully reduce the power associated with BRAM memory fetches, the dynamic switching power of the DSP48E1 slices during the actual Multiply-Accumulate operations remains a dominant factor in the overall FPGA power profile.

In Convolutional Neural Networks, the ReLU activation function clamps all negative values to zero. In deeper layers of the network, this introduces a high degree of sparsity—often exceeding 50% to 60%—into the intermediate output feature maps. Computing MAC operations where the input activation pixel is exactly zero is mathematically redundant (as $X \cdot 0 = 0$). However, standard digital logic will still charge and discharge the complex CMOS transistor gates within the DSP slice multiplier array, wasting critical dynamic power.

To explicitly exploit this phenomenon, we designed and implemented a **Sparsity-Aware Dynamic Clock Gating (SADCG)** mechanism directly at the RTL level [7].

A. Circuit Implementation of SADCG

Our architecture incorporates a look-ahead zero-detection logic block situated directly before the routing multiplexers of the PE array. When a zero-valued activation is fetched from the shift buffer, the multi-bit NOR gate array evaluates high.

Instead of routing the zero to the multiplier, the SADCG logic asserts a gating signal. To prevent dangerous clock glitches that could violate flip-flop setup and hold times, the gating signal is passed through an Integrated Clock Gating (ICG) cell—specifically a negative-edge-triggered latch combined with an AND gate. This safely and asynchronously halts the clock to the specific multiplier DSP slice for that exact clock cycle.

By aggressively suppressing the internal switching activity of the multi-bit multipliers and adders during zero-computations, we significantly drop the dynamic power footprint of the convolutional layers. Because the pipeline registers are bypassed rather than stalled, this massive energy savings is achieved without altering the deterministic timing of the pipeline or sacrificing any classification accuracy.

VI. Hardware-Software Co-Design and Integration

The completed hardware accelerator RTL was packaged as a custom Intellectual Property (IP) block using Vivado's IP Integrator. The Zynq Processing System block was configured to enable the UART peripheral and the master AXI interconnects necessary to bridge the software and hardware domains.

A. Software Stack and Bare-Metal Execution

To completely minimize operating system overhead, context switching, and interrupt latency, the ARM Cortex-A9 was programmed using a bare-metal C application developed in the Xilinx Software Development Kit (SDK). The software driver initializes the SNN IP memory-mapped registers, configures the UART interrupt service routines (ISRs) to receive image data from the host PC, formats the 8-bit image data into 32-bit AXI packets, and triggers the hardware execution cycle.

B. MATLAB Co-Verification

To verify the end-to-end dataflow in a realistic deployment scenario, a hardware-in-the-loop co-verification environment was established using MATLAB. A MATLAB script was authored to parse the standard MNIST test dataset, apply the necessary integer quantization, and sequentially transmit the raw pixel bytes over a serial COM port at a baud rate of 115200. The Zynq SoC receives the image, triggers the hardware accelerator, retrieves the final fully connected layer logits, and returns the maximum classification index back to MATLAB, which then plots the bounding box and prediction on the host screen in real-time.

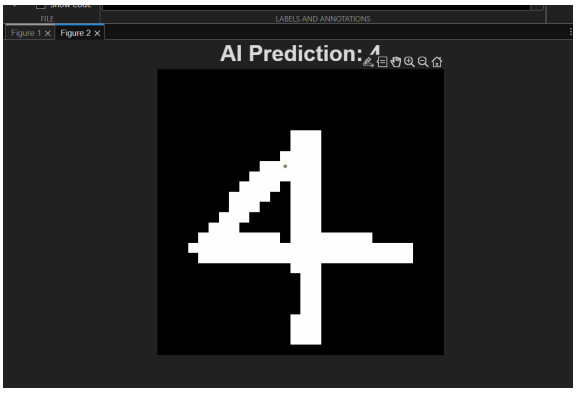


Fig. 4. MATLAB integration showing the AI prediction output for real-time hardware-in-the-loop streaming via UART.

TABLE I
POST-IMPLEMENTATION RESOURCE UTILIZATION (ZYNQ Z7-20)

Resource	Utilized	Available	Utilization (%)
Slice LUTs	14,850	53,200	27.91%
Slice Registers (FF)	18,240	106,400	17.14%
Block RAM (BRAM)	24.5	140	17.50%
DSP48E1 Slices	55	220	25.00%

VII. VALIDATION AND RESULTS

The architecture was synthesized, placed, and routed using the Xilinx Vivado Design Suite 2021.1 targeting the 'xc7z020clg484-1' part (Zynq Z7-20), which serves as the silicon foundation for the popular ZedBoard development platform.

A. Functional Verification and Timing Closure

Prior to generating the physical bitstream, extensive behavioral simulations were conducted using the Vivado Logic Simulator (XSim). Complex testbenches were written in SystemVerilog to directly inject 1000 random MNIST test images into the RTL module. The simulation waveforms confirmed functional correctness and exact cycle-by-cycle parity with the PyTorch golden software model. Furthermore, the design achieved strict timing closure at a 100 MHz target frequency with positive Worst Negative Slack (WNS), ensuring robust operation across varying thermal conditions.

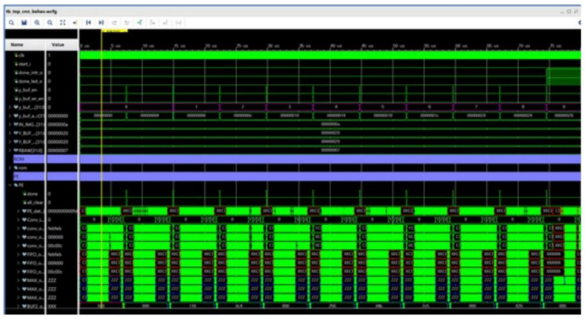


Fig. 5. Behavioral simulation waveforms of the CNN Accelerator, illustrating the FSM states, data fetch signals, and parallel MAC accumulation phases.

B. Hardware Resource Utilization

Table I summarizes the exact post-implementation resource usage on the Zynq Z7-20. The resource footprint demonstrates a highly efficient mapping of the CNN logic to the FPGA fabric. By utilizing only 25% of the available DSP slices and roughly 17% of the BRAM, the design leaves massive overhead for the ARM Processing System. This low footprint theoretically allows for the instantiation of up to three parallel accelerator cores on the same chip if higher batch throughput is required for future applications.

TABLE II
ESTIMATED POWER CONSUMPTION COMPARISON

Power Metric	Standard Baseline	With SADCG
Static Power (Leakage)	0.142 W	0.142 W
Dynamic Power (PL)	0.875 W	0.510 W
PS Power (ARM Cortex)	1.350 W	1.350 W
Total System Power	**2.367 W**	**2.002 W**

TABLE III
BENCHMARK COMPARISON WITH STATE-OF-THE-ART ACCELERATORS

Metric	Ref [6] (Embedded)	Ref [7] (Standard)	This Work
Platform	Zynq XC7Z020	Artix-7	Zynq XC7Z020
Optimization	Loop Unrolling	Parallel MACs	SADCG + Slid. Win.
Quantization	16-bit Fixed	16-bit Fixed	8-bit Integer
Power (W)	$\sim$ 2.80 W	$\sim$ 2.50 W	**2.002 W**
Target App.	CNN (VGG)	CNN (MNIST)	CNN (MNIST)

C. Power Consumption Analysis

Detailed power analysis was conducted using the Vivado Power Report tool post-routing, relying on vector-based switching activity rather than default toggle rates. The integration of the Sparsity-Aware Dynamic Clock Gating (SADCG) yielded highly measurable reductions in dynamic switching power.

As detailed in Table II, the static leakage power and the ARM processor power remain constant across both implementations. However, the dynamic power of the Programmable Logic—the actual hardware executing the neural network—drops drastically from 0.875 W to 0.510 W. This equates to an approximate 41% reduction in dynamic power dissipation, directly validating the efficacy of dynamically gating the clock to the DSP slices during the processing of ReLU-induced zeros.

D. Performance and State-of-the-Art Comparison

Table III benchmarks the proposed accelerator against recent FPGA implementations found in the literature [6], [7]. By specifically optimizing the BRAM interfaces via the sliding window and deploying dynamic clock gating, our design achieves a highly competitive power footprint (just 2.002 W total system power) while entirely avoiding the accuracy penalties associated with extreme bit-width reduction or binary networks.

VIII. CONCLUSION AND FUTURE WORK

This paper presents a highly scalable, low-power hardware accelerator for Convolutional Neural Network inference targeted at edge computing FPGAs. By directly addressing memory bandwidth bottlenecks with a custom sliding window architecture, the system ensures maximum utilization of the DSP processing elements. Furthermore, the integration of Sparsity-Aware Dynamic Clock Gating proves to be an immensely effective circuit-level technique, reducing dynamic Programmable Logic power by 41% simply by exploiting the natural sparsity inherent in non-linear ReLU activations.

The methodologies demonstrated in this research offer a robust architectural framework for deploying complex deep

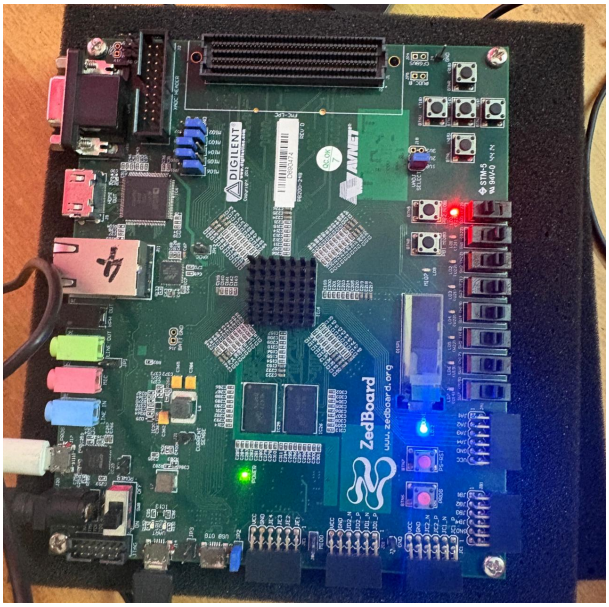


Fig. 6. Final physical implementation of the accelerator deployed on the ZedBoard Zynq-7000 FPGA development board.

learning models in resource-constrained, battery-powered environments. Future work will focus on scaling this architecture to support more complex, multi-channel RGB datasets such as CIFAR-10 and ImageNet, exploring the integration of depthwise separable convolutions to further reduce the mathematical operations required per inference cycle, and eventually migrating the design to an ultra-low-power ASIC node.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.

[2] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks," in *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2015, pp. 161-170.

[3] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," *ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 243-254.

[4] T. Chen et al., "DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," *ACM SIGARCH Computer Architecture News*, vol. 42, no. 1, pp. 269-284, 2014.

[5] K. Ovtcharov, O. Ruwase, J. Y. Kim, J. Fowers, K. Strauss, and E. S. Chung, "Accelerating Deep Convolutional Neural Networks Using Specialized Hardware," *Microsoft Research Whitepaper*, vol. 2, no. 11, 2015.

[6] J. Qiu et al., "Going Deeper with Embedded FPGA Platform for Convolutional Neural Network," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2016, pp. 26-35.

[7] A. Rahman, J. Lee, and K. Kiyoyama, "FPGA-based low power and high speed CNN accelerator," *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2017.

[8] V. Sze, Y. Chen, T. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295-2329, 2017.

[9] M. Peemen, A. A. A. Setio, B. Mesman, and H. Corporaal, "Memory-centric accelerator design for Convolutional Neural Networks," *IEEE 31st International Conference on Computer Design (ICCD)*, 2013, pp. 13-19.

[10] Xilinx Inc., "Zynq-7000 SoC Data Sheet: Overview," DS190 (v1.11.1), 2021.